

COSC 81/281: Problem Set 2: Learning, Optimization, and Control

Due Monday, October 5 (11:59pm ET)

Problem sets are graded **pass/fail on completion**, and there are no late days. The problem sets are where the learning happens, and they are the best available practice for the midterm and final exam. Submit to **Gradescope** (<https://www.gradescope.com/courses/1372528>) and assign each question to a page in your submission. Questions are welcome on **Slack**. Starred (★) questions are required for COSC 281 students and optional (ungraded) for COSC 81 students.

AI Policy. You may use any AI tools you like to assist you with your assignments, and you should, as that is likely how you will work for the rest of your career. That being said, the written portions are practice for the in-class midterm and exam, which are closed book and on paper. You should strive to answer these yourself and use AI as a tutor and not an oracle.

Please include an AI Tool Disclosure Below

Overview

This problem set covers forward kinematics, learning from experience, gradient descent, and configuration space. Implement gradient descent and examine its sensitivity to the learning rate. Complete P1–P6 and the programming task. Starred subparts are required for COSC 281 students and optional, ungraded practice for COSC 81 students.

What to submit. Submit to **Gradescope** (<https://www.gradescope.com/courses/1372528>):

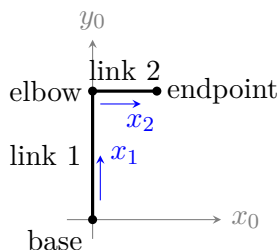
1. **Written work (PDF):** your answers to Part 1 and every plot, measurement, and short response requested in Part 2. You may write on this handout or compile `ps2/handout.tex`, placing answers and figures in its `studentanswer` environments. Typed boxes expand as needed. Assign each question to its page or pages when you upload.
2. **Code:** `gd.py`. Before submission, run the public checks from the repository root.
`python ps2/code/autograder.py`

Part 1: Written problems

Show your work in each response area. No programming is needed for Part 1.

P1: Forward kinematics of a two-link arm

A planar arm lies in the world xy -plane, with link lengths 2 and 1. The first joint angle is 90° about $+z$; the second is -90° relative to the first link. Frame 0 is the world frame. Frame 1 has its origin at the base and its x -axis along the first link. Frame 2 has its origin at the elbow and its x -axis along the second link. The diagram shows the configuration, not additional data.



The notation ${}^a\mathbf{T}_b$ maps coordinates from frame b into frame a . Use column vectors and the supplied homogeneous transforms:

$${}^0\mathbf{T}_1 = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad {}^1\mathbf{T}_2 = \begin{bmatrix} 0 & 1 & 0 & 2 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

- (a) Compute ${}^0\mathbf{T}_2$ and the world coordinates of the endpoint, whose frame-2 homogeneous coordinates are $(1, 0, 0, 1)^\top$.

Answer / work

- (b) Explain why ${}^1\mathbf{T}_2 {}^0\mathbf{T}_1$ is not the required composition. Refer to the input and output frames, not only to noncommutativity.

Answer / work

P2: Learning the value of a fixed policy

A robot follows a fixed policy π . Two recorded episodes are shown below. Each arrow is labeled by the reward received on that transition; T is terminal and $V^\pi(T) = 0$. Use $\gamma = \frac{1}{2}$.

$$\text{Episode 1: } A \xrightarrow{0} B \xrightarrow{4} T, \quad \text{Episode 2: } A \xrightarrow{0} B \xrightarrow{2} T.$$

The policy remains fixed throughout this question. Direct evaluation estimates a state's value by averaging the observed discounted returns from that state. Temporal-difference (TD) evaluation instead updates after each transition using

$$\text{sample} = r + \gamma V^\pi(s'), \quad V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha \text{sample}.$$

For a transition into T , the sample is just r .

- (a) Compute the discounted return from A and from B in each episode. Then compute the **direct-evaluation** estimates of $V^\pi(A)$ and $V^\pi(B)$.

Answer / work

- (b) Initialize both state values to zero and use $\alpha = \frac{1}{2}$ to do **TD learning**. Process Episode 1 followed by Episode 2, in the order shown. For each of the four transitions, record the sample and the updated value of the state being left. Use the current estimates at each step.

Answer / work

- (c) ★ (required for 281; optional for 81) Explain why the TD estimates need not equal the direct-evaluation estimates after these two episodes. Does either calculation establish that π is optimal? Explain in at most three sentences.

Answer / work

P3: Exploration and observed rewards

A tabular Q-learning agent operates on a large grid. The goal is 22 steps from the start and gives reward +100 on entry, ending the episode. A charging pad adjacent to the start gives reward +1 each time it is entered. All other rewards are zero. Episodes end after at most 50 steps, $\gamma = 0.9$, $\alpha = 0.5$, and the Q-table is initialized to zero. The agent uses ϵ -greedy exploration with $\epsilon = 0.5$ for the first 100 episodes and $\epsilon = 0$ thereafter. After 100,000 episodes, the goal has never been reached. Q-values near the pad are positive, while almost all other entries remain zero.

- (a) Explain this outcome in at most three sentences. Account for both the positive values near the charging pad and the absence of the goal reward from the learned Q-values.

Answer / work

- (b) Would increasing the learning rate alone resolve this problem? Explain your answer.

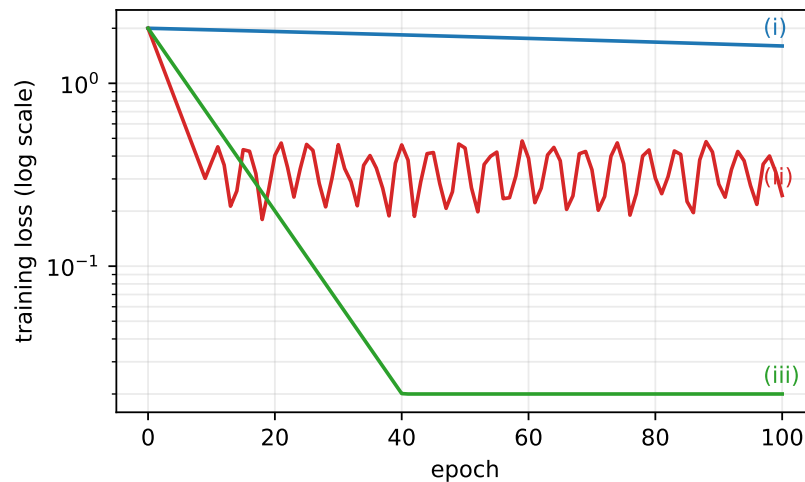
Answer / work

- (c) Propose one change that would improve the opportunity to discover the goal. Explain how it addresses the mechanism identified in part (a).

Answer / work

P4: Learning-rate effects

The three curves below come from the polynomial fit of programming Task 1: same data, same initialization, three different learning rates, α . None of them is labeled.



- (a) For α , match each of (i), (ii), (iii) to one of: too small, well chosen, too large but not diverging. Briefly justify each match.

Answer / work

- (b) For each unsuitable learning rate, describe an appropriate adjustment.

Answer / work

- (c) ★ (required for 281; optional for 81) There is a fourth regime none of these curves shows. Name it, describe what its loss curve looks like, and state which of the three learning rates here is closest to producing it.

Answer / work

P5: A quadratic approximation

Construct a quadratic Taylor approximation, then apply gradient descent to that approximation. Determine the learning rate that reaches its minimizer in one step.

Let $f(x) = \cos(x) - \sin(x)$.

Hint: $\sin'(x) = \cos(x)$, $\cos'(x) = -\sin(x)$, $\sin(\frac{\pi}{2}) = 1$, and $\cos(\frac{\pi}{2}) = 0$.

- (a) Build the quadratic Taylor approximation $\hat{y}(x)$ of f at $x_0 = \frac{\pi}{2}$. Show the three evaluations. You do not need to expand or simplify.

Answer / work

- (b) Where is the minimizer x^* of $\hat{y}$? Justify with $\hat{y}'(x^*) = 0$.

Answer / work

- (c) Take one gradient descent step *on $\hat{y}$* with learning rate $\alpha = \frac{1}{2}$, starting from $x_k = \frac{\pi}{2} - 1$. Compute the updated point and compare its distance from x^* with the initial distance.

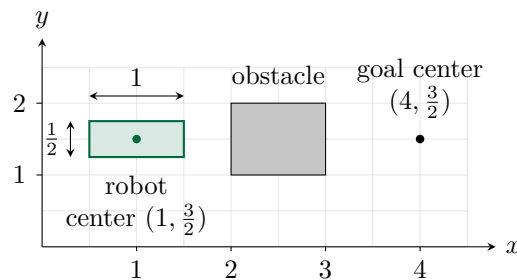
Answer / work

- (d) ★ (required for 281; optional for 81) Find the learning rate α^* for which one step from this same x_k lands *exactly* on x^* . Take the step to confirm it. Then show that with α^* the landing point does not depend on x_k at all, and name the classical method you have just rediscovered.

Answer / work

P6: From workspace to configuration space

A rectangular robot must move through the workspace shown below while avoiding the shaded obstacle, which occupies $[2, 3] \times [1, 2]$. The robot is 1 unit wide and $\frac{1}{2}$ unit tall, and we describe its position by the coordinates of its center. Throughout parts (b) and (c), the robot may translate but may not rotate: its edges remain parallel to the world axes. Contact with the obstacle counts as a collision. The axes show a portion of an otherwise unbounded workspace.



- (a) Specify the configuration coordinates and the dimension of the configuration space. If rotation were also allowed, what coordinate would have to be added?

Answer / work

- (b) With the robot restricted to translation at the fixed orientation shown, express the problem in configuration space and compute the set of center positions for which the robot is in collision. Explain how each of its four boundaries follows from the dimensions of the robot and obstacle. Would checking the center against the original obstacle suffice?

Answer / work

- (c) Still without rotation, both center configurations $(1, \frac{3}{2})$ and $(4, \frac{3}{2})$ are collision-free. Is the straight-line motion between them valid? Explain what this example requires a motion planner to check in addition to the endpoints of an edge.

Answer / work

Part 2: Programming

The starter code is in `ps2/code/`. Task 1 implements gradient descent. Run `python ps2/code/autograder.py` from the repository root to check the documented function interfaces.

Task 1: gradient descent from scratch

Connection to Part 1. P4 asks you to interpret learning-rate behavior. This task implements gradient updates for a four-parameter model. P5 examines gradient steps on a local quadratic approximation.

Implement the gradient and update directly, without automatic differentiation or optimizer libraries. You will fit a degree-3 polynomial $\hat{y}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$ to a provided noisy 1D dataset (40 points from a smooth curve that is *not* a polynomial, so the fit cannot be perfect) by minimizing the mean squared error

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N (\hat{y}(x_i) - y_i)^2.$$

In `gd.py`, implement the loss, its analytic gradient with respect to $\boldsymbol{\theta}$ (derive it on paper first, using the fact that the model is linear in $\boldsymbol{\theta}$), and the gradient descent loop $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla \mathcal{L}(\boldsymbol{\theta})$ that records the loss at every iteration.

Then run the learning-rate experiment. Sweep α over the four values in the starter config, from small to large, and plot all four loss curves on one set of axes with a log-scale y axis. Deliver the plot of the fitted curve against the data for your best α , the four-curve loss plot, and three sentences classifying each α (too small, well chosen, too large but stable, diverging) with the evidence from your plot.

Submission. Put the four implementations in `gd.py`. Put `ps2_fit.png`, `ps2_loss_curves.png`, and your learning-rate classification in the PDF response below.

PDF responses.

- (a) Include `ps2_fit.png`, showing the data and fitted polynomial for your selected learning rate.

Answer / work

- (b) Include `ps2_loss_curves.png`, with all four learning rates labeled and a logarithmic loss axis.

Answer / work

- (c) Classify the observed behavior of each learning rate and justify the classification using the curves.

Answer / work